Project 2: Multithreaded Collatz

Nathan Garrett and Leaf Wood | COP4634

Introduction

For this project, we tested how using multiple threads changes the time it takes to run the Collatz problem. If a number is even, we divide it by 2. If it is odd, we do 3n + 1. We keep doing this until the number reaches 1. We count the steps and put them into a histogram.

Our program uses a shared counter to give each thread a number to work on. We used two mutex locks. One protects the counter and the other protects the histogram. The threads do the Collatz calculations without holding those locks. We can also run the program with **-nolock** to turn both locks off.

How we tested it

We tested the program on a Windows laptop and the university Linux server. We used **N = 10,000,000** with 1 through 8 threads. We did 10 runs with locks and 10 without locks for each thread count. That was 160 timed runs on each computer, or **320 runs total**.

We ran **make experiment** to collect our results. The program times the work from just before creating the threads until they finish. It does not count the time spent printing the histogram. Our script saved the times and calculated the averages. We also saved a locked one-thread histogram from each computer.

Computers we used

Computer	**Windows laptop**	**University Linux server**
CPU	AMD Ryzen 5 7640HS	Intel Xeon Gold 6338N, 2.20 GHz
Processors	6 cores, 12 logical processors	48 logical processors reported
Memory	15.19 GiB	125 GiB
System	Windows 11 Home	Linux, kernel 4.18.0
Compiler	MSYS2 g++ 16.2.0	GCC 8.5.0

Results

We averaged 10 runs for each setting. These graphs show how long the program took on each computer. Lower bars mean shorter times.

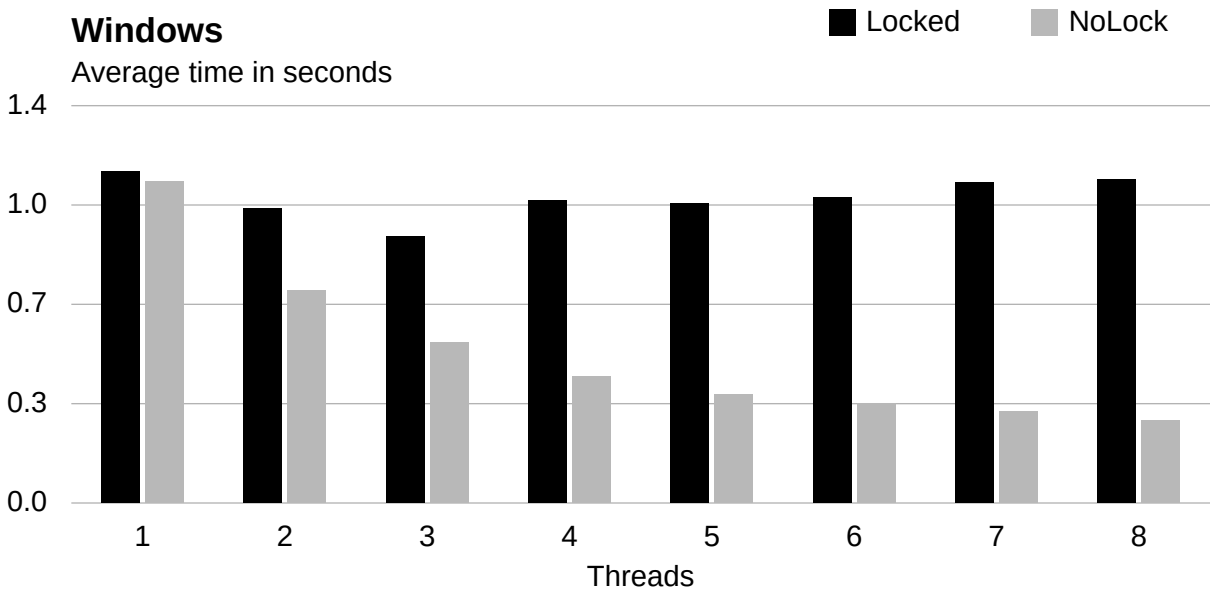

Figure 1. Average times on Windows with N = 10,000,000.

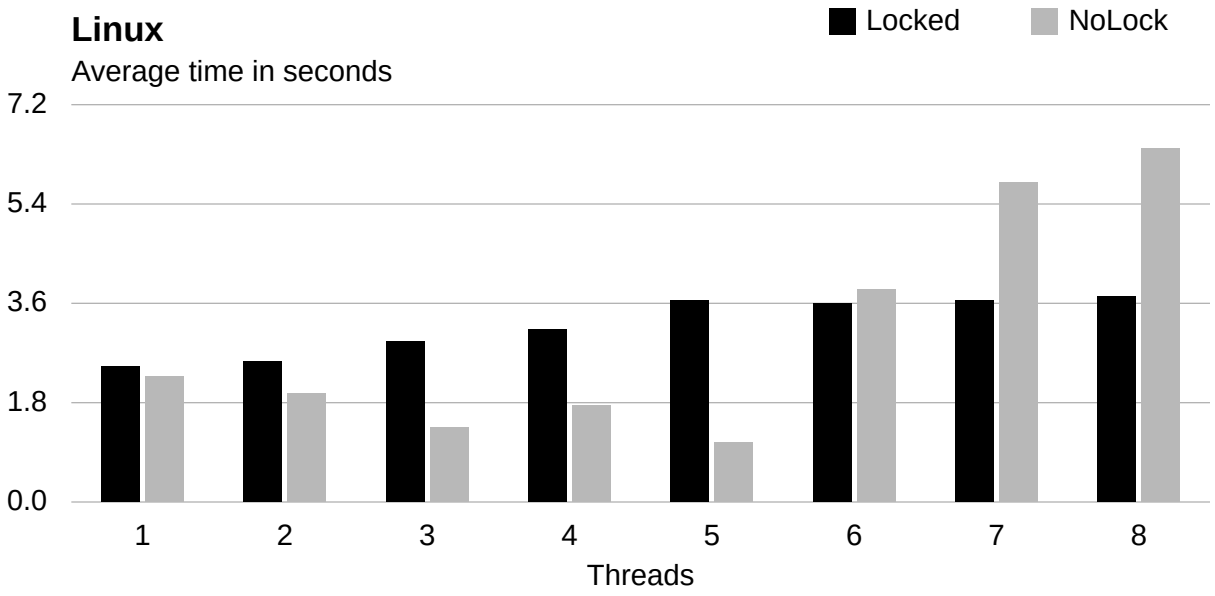

Figure 2. Average times on the Linux server with N = 10,000,000.

Results (continued)

Here are the average times from both computers. All times are in seconds.

Threads	Windows Locked	Windows NoLock	Linux Locked	Linux NoLock
1	1.1694	1.1330	2.4525	2.2816
2	1.0369	0.7502	2.5476	1.9711
3	0.9389	0.5670	2.9011	1.3535
4	1.0656	0.4448	3.1180	1.7523
5	1.0550	0.3831	3.6489	1.0841
6	1.0776	0.3495	3.5963	3.8422
7	1.1303	0.3237	3.6541	5.7883
8	1.1414	0.2916	3.7166	6.4021

Table 1. Average of 10 runs for each thread count.

Histogram

We also compared the locked one-thread histograms from Windows and Linux. They matched. Each histogram counted 10,000,000 starting values.

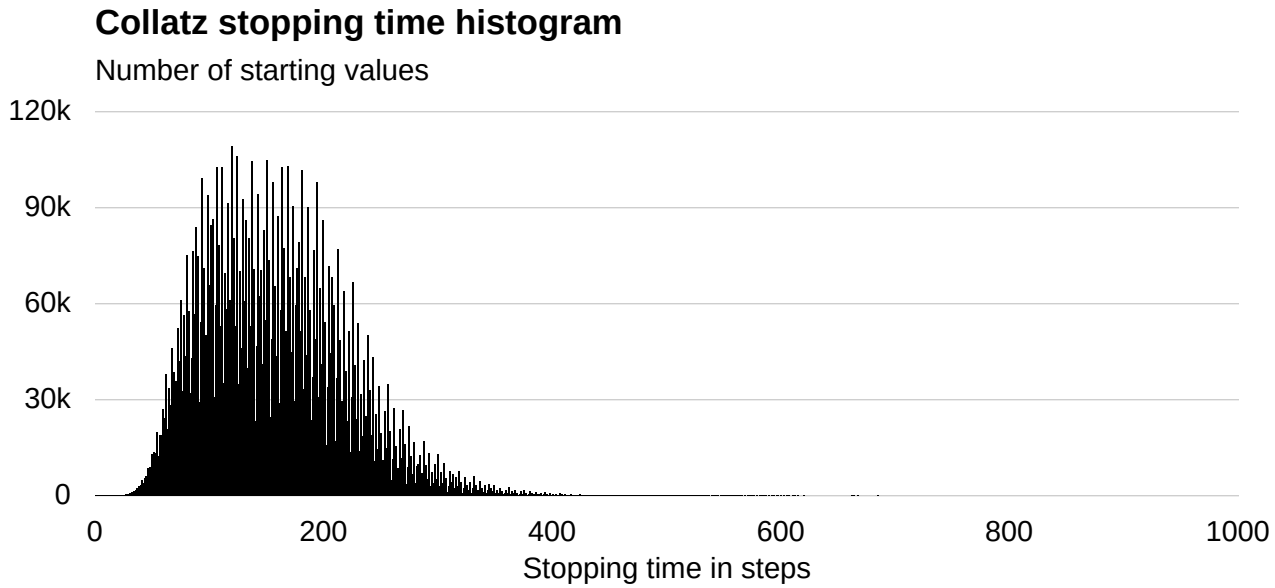

Figure 3. The saved locked one-thread histogram. Both computers gave the same result.

Extra checks

We ran a smaller test with N = 500,000. This let us compare the results with five threads against the one-thread result.

Computer	Threads and mode	Histogram total	Compared to 1 thread
Windows	1 / NoLock	500,000	Reference
Windows	5 / NoLock	545,636	332 bins different
Windows	5 / Locked	500,000	All bins matched
Linux	1 / NoLock	500,000	Reference
Linux	5 / NoLock	665,894	360 bins different
Linux	5 / Locked	500,000	All bins matched

Table 2. Results from the separate 500,000-number checks.

Discussion

On Windows, **3 threads** gave us the fastest average time with locks. It took about **0.939 seconds**. One thread took about 1.169 seconds. That is around 20% faster.

Linux was different. Its fastest locked run was with **1 thread**, at about 2.452 seconds. With 8 threads, it took about 3.717 seconds. So more threads did not always help.

On Windows, turning off the locks made the runs faster as we added threads. But the five-thread NoLock checks did not give the same histogram as the one-thread checks on either computer. With locks turned on, the histograms matched.

We think the locks added some waiting because multiple threads were trying to update the same counter and histogram. The two computers also have different hardware, which helps explain why their times were not the same.

Conclusion

We got the program running on both computers and recorded all 320 timed runs. The biggest thing we learned is that adding threads does not automatically make a program faster. The locks slowed down some runs, but our locked check results matched.

Sources: Project 2 MT Collatz.pdf, Sample Report (2).pdf, and our saved Windows and Linux results.

AI disclosure: AI helped us edit the report and with Excel automation. We collected the measurements by running our program.